Name: Akshit Kumar

SAP ID: 590020087

Experiment 7

Experiment: Functions

---

1. Write a Python function to find the maximum and minimum numbers from a sequence of numbers.

Code:

In [1]:
```
def minimumMaximum(numbers):
    if not numbers:
        return None, None

    maximumValue = numbers[0]
    minimumValue = numbers[0]

    for num in numbers[1:]:
        if num > maximumValue:
            maximumValue = num
        elif num < minimumValue:
            minimumValue = num

    return maximumValue, minimumValue

nums = [12, 5, 25, 1, 9]
maximumNumber, minimumNumber = minimumMaximum(nums)
print(f"Maximum: {maximumNumber}, Minimum: {minimumNumber}")
```

```
Maximum: 25, Minimum: 1
```

Explanation

1. A function named `minimumMaximum()` is defined which takes a list of numbers as input.
2. The condition `if not numbers` checks whether the list is empty.
   - If the list is empty, the function returns `None, None`.
3. The first element of the list is assigned to both `maximumValue` and `minimumValue` as an initial reference.
4. A loop iterates through the remaining elements of the list starting from the second element.
5. For each number:
   - If the number is greater than `maximumValue`, it updates the maximum value.
   - If the number is smaller than `minimumValue`, it updates the minimum value.
6. After checking all elements, the function returns both maximum and minimum values.
7. A list `nums` is created with sample values.
8. The function is called with this list, and the returned values are stored in `maximumNumber` and `minimumNumber`.
9. Finally, the maximum and minimum values are printed using formatted output.

---

2. Write a python function that takes a positive interger and returns the sum of the cube of all the positive integers smaller than the specified number.

Code:

In [2]:
```
def sumOfCube(n):
    if n <= 1:
        return 0

    totalSum = 0
    for i in range(1, n):
        totalSum += i ** 3

    return totalSum

number1 = 12
result1 = sumOfCube(number1)

print(f"The sum of the cube of all positive integers smaller than {number1} is {result1}")

number2 = 9
result2 = sumOfCube(number2)
print(f"The sum of the cubes of all positive integers smaller than {number2} is {result2}")
```

```
The sum of the cube of all positive integers smaller than 12 is 4356
The sum of the cubes of all positive integers smaller than 9 is 1296
```

Explanation

1. A function named `sumOfCube()` is defined which takes a positive integer `n` as input.
2. The condition `if n <= 1` checks whether the number is less than or equal to 1.
   - If true, the function returns 0 because there are no positive integers smaller than 1.
3. A variable `totalSum` is initialized to 0 to store the sum of cubes.
4. A loop runs from 1 to `n-1` using `range(1, n)`, since we need all positive integers smaller than `n`.
5. Inside the loop, the cube of each number (`i ** 3`) is calculated and added to `totalSum`.
6. After the loop finishes, the function returns the total sum of cubes.
7. The function is called with two different values (`12` and `9`).
8. The results are stored in variables and printed using formatted output.

---

3. Write a python function to print 1 to n using recursion.

Code:

In [3]:
```
def printNumbers(n, current = 1):
    if current > n:
        return

    print(current)

    printNumbers(n, current + 1)

printNumbers(5)
```

```
1
2
3
4
5
```

Explanation

1. A function named `printNumbers()` is defined with two parameters:
   - `n`: the limit up to which numbers are printed.
   - `current`: the current number to print (default value is 1)
2. The condition `if current > n` acts as the base case of recursion.
   - When the current value exceeds `n`, the function stops calling itself.
3. The current value is printed using the `print()` function.
4. The function calls itself recursively with `current + 1`, moving to the next number.
5. This process continues until the base condition is met.
6. Finally, the function is called with `n = 5`, which prints numbers from 1 to 5.

---

4. Write a Python function to print 1 to n using recursion.

Code:

In [ ]:
```
def printNumbers(n):
    if n == 0:
        return

    printNumbers(n - 1)  # recursive call
    print(n)             # print after recursion

printNumbers(5)
```

```
0
1
1
2
3
5
8
13
21
34
55
89
144
233
377
610
987
1597
2584
4181
```

Explanation

1. A function named `printNumbers()` is defined which takes an integer `n` as input.
2. The condition `if n == 0` is the base case of recursion.
   - When `n` becomes 0, the function stops calling itself.
3. The function first calls itself with `n - 1`, which helps in reaching the base case.
4. After the recursive call, the value of `n` is printed.
5. This ensures that numbers are printed in ascending order from 1 to `n`.
6. Finally, the function is called with value `5`, so numbers from 1 to 5 are printed.

---

5. Write a lambda function to find volume of cone.

Code:

In [ ]:
```
import math

volume = lambda r, h: (1/3) * math.pi * (r ** 2) * h

r = 5
h = 10

print(f"The volume is {volume(r, h)}")
```

```
The volume is 261.7993877991494
```

Explanation

1. The `math` module is imported to use the constant value of π (pi).
2. A lambda function named `volume` is created which takes two parameters: radius (`r`) and height (`h`).
3. The lambda function calculates the volume of a cone using the formula:
   Volume = (1/3) × π × r² × h
4. Values for radius (`r = 5`) and height (`h = 10`) are assigned.
5. The lambda function is called with these values to compute the volume.
6. The result is displayed using formatted output.

---

6. Write a lambda function which gives tuple of max and min from a list.

Code:

In [6]:
```
getMinimumMaximum = lambda lst: (max(lst), min(lst))

numbers = [12, 5, 25, 15]
result = getMinimumMaximum(numbers)

print(result)
```

```
(25, 9)
```

Explanation

1. A lambda function named `getMinimumMaximum` is defined which takes a list `lst` as input.
2. The lambda function returns a tuple containing:
   - The maximum value using `max(lst)`.
   - The minimum value using `min(lst)`.
3. A list `numbers` is created with sample values.
4. The lambda function is called with this list, and the result is stored in the variable `result`.
5. The output is a tuple containing both maximum and minimum values.
6. Finally, the result is printed.

---

7. Write a function to explain mentioned concepts:

1. Keyword argument
2. Default argument
3. Variable length argument

a.) Keyword argument

In [7]:
```
def describePet(animalType, petName):
    print(f"I have a {animalType} named {petName}.")

describePet(petName = "Jacky", animalType = "Dog")
```

```
I have a Dog named Jacky.
```

b.) Default argument

In [8]:
```
def greetFunction(name, message = "Hello"):
    print(f"{message}, {name}")

greetFunction("Akshit")

greetFunction("Aash", "Hello")
```

```
Hello, Akshit
Hello, Aash
```

c.) Variable length argument

In [9]:
```
def sumOfNumbers(*numbers):
    return sum(numbers)

print(sumOfNumbers(9, 12, 25, 15))
```

```
61
```

Explanation

This program demonstrates three types of function arguments in Python: keyword arguments, default arguments, and variable-length arguments.

(a) Keyword Argument

1. A function `describePet(animalType, petName)` is defined with two parameters.
2. The function prints a sentence using both parameters.
3. While calling the function, arguments are passed using parameter names:
   `petName = "Jacky"` and `animalType = "Dog"`.
4. This is called a keyword argument because the order of arguments does not matter when names are specified.

(b) Default Argument

1. A function `greetFunction(name, message = "Hello")` is defined.
2. The parameter `message` has a default value `"Hello"`.
3. When the function is called with only one argument, the default value is used.
4. When both arguments are provided, the given value overrides the default value.

(c) Variable-Length Argument

1. A function `sumOfNumbers(*numbers)` is defined using `*numbers`.
2. This allows the function to accept any number of arguments.
3. The arguments are stored as a tuple.
4. The built-in `sum()` function is used to calculate the total.
5. The result is returned and printed.